实验 10　内存管理算法

10.1 实验目标

(1)实测分析局部性好坏对程序性能的影响

(2)掌握虚拟/物理地址映射并在 Linux 中验证

(3)掌握连续内存分配的方法并进行模拟

(4)掌握页面置换算法原理并进行模拟分析

10.2 预备知识

10.2.1 虚拟地址空间

虚拟地址空间[1]就是程序可以使用的虚拟地址的有效范围。虚拟地址与物理地址的映射关系由操作系统决定，且虚拟地址空间大小也由操作系统决定，同时还受编译模式影响。在 64 位编译模式下，一个指针或地址占用 8 字节内存，共有 64 位，理论上能够访问的虚拟地址空间大小为 2^{64}（**数字很大！，实际没必要**）。实际上，Windows 和 Linux 都对虚拟地址进行了限制，仅使用虚拟地址低 48 位（6 个字节），总的虚拟地址空间大小为 2^{48}。

此外，CPU 支持的物理内存只是理论上的数据，实际应用会受操作系统的限制，如 Win7 64 位家庭版最大仅支持 8GB 或 16GB 物理内存、Win7 64 位专业版或企业版能够支持到 192GB 的物理内存。

10.2.2 内存管理

计算机不可能将所有用户进程和系统所需要的全部程序和数据装入主存，因此操作系统需对内存空间进行合理划分和动态分配。内存管理的概念就是操作系统对内存的划分和动态分配。内存管理的功能包括：①内存空间分配与回收；②地址转换；③内存空间的扩充；④存储保护。

连续分配存储管理方式是最早出现的一种存储器分配方式。该分配方式为一个用户程序分配一个连续的内存空间，即程序中代码或数据的逻辑地址相邻，体现在内存分配时物理地址的相邻。动态分区存储管理是连续分配存储管理方式的

[1] https://www.cnblogs.com/zjuhaohaoxuexi/p/16101623.html

一种，它是指在处理作业过程中划分分区，使分区大小正好适合作业的需要，而且分区的个数是可变的。当要装入一个作业时，根据作业需要的内存量，查看是否有足够的空闲空间。若有，则按需求量分割一部分给作业；若无，则作业等待。作业完成，所占有的存储空间需回收。动态分区分配会产生外部碎片，克服外部碎片可以采用紧凑技术来解决，即操作系统不时地对进程进行移动和整理。但这需要重定位寄存器的支持，且相对费时。

现代操作系统中，为提高内存利用率，提供了内外存进程对换机制，内存空间的分配和回收均以页为单位进行。一个进程只需将其一部分（段或页）调入内存便可运行，还支持请求调页的存储管理方式。当进程在运行中需要访问某部分程序和数据时，发现其所在页面不在内存，就立即提出请求（向 CPU 发出缺中断），由系统将其所需页面调入内存。这种页面调入方式叫请求调页。当 CPU 接收到缺页中断信号，中断处理程序先保存现场，分析中断原因，转入缺页中断处理程序。中断处理程序通过查找页表，得到该页所在外存的物理块号。如果此时内存未满，能容纳新页，则启动磁盘 I/O 将所缺之页调入内存，然后修改页表。如果内存已满，则须按某种置换算法从内存中选出一页换出，是否重新写盘由页表的修改位决定，然后将缺页调入，修改页表。利用修改后的页表，去形成所要访问数据的物理地址，再去访问内存数据。整个页面的调入过程对用户是透明的。

Windows 系统观察进程页面错误数：打开【任务管理器】，单击【详细信息】选项卡，右键单击列名，在弹出的窗口中单击【选择列】，然后选中【页面错误】选项，单击【确定】即可。在任务管理器的【详细信息】中可看到进行缺页次数的列。**Linux 系统查看进程页面错误数：ps -o min_flt,maj_flt 1538**，maj_flt 表示 major page faults(只能通过访问磁盘来解决)，min_flt 表示 minor page faults(可通过共享内存中已有页面来解决)。

10.3 实验步骤

10.3.1 程序局部性分析

实验需要验证 4 点：①小数组、局部性好，访问数组时会将部分页面调到内存中，**占用内存增加、页面错误增加**；②小数组、局部性差，局部性差的情况需要消耗更多的时间；③页面被调入内存后，再次访问，效率会提高。

实验操作过程：

①打开 Finalshell，创建一个 C 程序文件，命名为 **access_page.cpp** 并打开：

```
[root@localhost ~]# access_page.cpp
[root@localhost ~]# vi access_page.cpp
```

②按 **i** 键进入 **insert** 模式，在 access_page.cpp 文件中写入如下代码，并按 Esc 键退出 **insert** 模式，输入**:wq** 保存：

```
#include <iostream>
#include <ctime>
#include <sys/types.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdint.h>
using namespace std;
int MyArray[10240][20480];
const string str[4] = { "局部性好","局部性差","2048*2048","10240*20480" };
int main() {
    int pid = getpid();
    cout << "当前进程的 PID 是"<<pid<<"，您可以根据进程号查看进程。"<<endl;
    cout << "模式(1. 局部性好；2. 局部性差)" << endl;
    cout << "数组大小(1. 2048*2048；2. 10240*20480)" << endl << endl;
    int op1 = 0, op2 = 0;
    while (1) {
        cout << "*******  请输入组合  *******" << endl;
        cin >> op1 >> op2;
        cout  <<  "您选择了组合("<<str[op1-1]<<","<<str[op2+1]<<")，运行时间为：";
        int sizex=1, sizey=1;
        if (op2 == 1) {
            sizex = sizey = 2048;
        } else if (op2 == 2) {
            if (op1 == 1) {
                sizex = 10240;
                sizey = 20480;
            } else if (op1 == 2) {
                sizey = 10240;
                sizex = 20480;
            }
        }
        clock_t start, end;
        start = clock();
        for (int i = 0; i < sizex; i++)
            for (int j = 0; j < sizey; j++) {
                if (op1 == 1) {
```

```
                    MyArray[i][j] = 0;
                } else if(op1==2) {
                    MyArray[j][i] = 0;
                }
            }
        end = clock();
        double time1 = (double)(end - start) / CLOCKS_PER_SEC;
        cout << time1 << "秒" << endl << endl;
    }
    return 0;
}
```

③编译 access_page.cpp 文件，生成可执行文件 access_page，并执行：

```
[root@localhost ~]# g++ access_page.cpp -o access_page
[root@localhost ~]# ./access_page
```

上述可不断地循环测试，请选择不同局部性好还和数组大小组合，分析输出结果。

10.3.2 虚拟/物理地址验证

Linux 的**/proc/<PID>/pagemap** 文件[2]允许用户查看当前进程虚拟页的物理地址等相关信息。用户态程序可以根据该文件计算出虚拟页映射到哪个物理页。实验输出进程中某个或多个全局变量或自定义函数的虚拟地址、所在页号、所在物理页框号、物理地址等信息。

实验操作过程：

①打开 Finalshell，创建一个 C 程序文件，命名为 **address.c** 并打开：

```
[root@localhost ~]# address.c
[root@localhost ~]# vi address.c
```

②按 **i** 键进入 **insert** 模式，在 address.c 文件中写入如下代码，并按 Esc 键退出 **insert** 模式，输入**:wq** 保存：

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdint.h>
```

[2] https://geneblue.github.io/2017/05/10/android/sec--android-pagemap/

```
char buf[200];
//计算虚拟地址对应的地址，传入虚拟地址 vaddr
void mem_addr(char* str, unsigned long pid, unsigned long vaddr, unsigned long* paddr) {
    int pageSize = getpagesize();
    unsigned long v_pageIndex = vaddr / pageSize;
    unsigned long v_offset = v_pageIndex * sizeof(uint64_t);// 计算在/proc/pid/page_map 文件中的偏移量
    unsigned long page_offset = vaddr % pageSize;//计算虚拟地址在页面中的偏移量
    uint64_t item = 0;
    sprintf(buf, "%s%lu%s", "/proc/", pid, "/pagemap");
    int fd = open(buf, O_RDONLY);
    lseek(fd, v_offset, SEEK_SET);
    read(fd, &item, sizeof(uint64_t));
    uint64_t phy_pageIndex = (((uint64_t)1 << 55) - 1) & item;//计算物理页号，即取 item 的 bit0-54
    *paddr = (phy_pageIndex * pageSize) + page_offset;//再加上页内偏移量就得到了物理地址
    printf("[%s] pid = %lu, virtual addr = 0x%lx, page no. = 0x%lx, phy addr = 0x%lx, phy no. = 0x%lx\n", str, pid, vaddr, v_pageIndex, *paddr, phy_pageIndex);
    sleep(1);
}

const int a = 100;
int main() {
    int b = 100;
    static int c = 100;
    const int d = 100;
    unsigned long phy = 0;
    char *p = (char*)malloc(100);
    int pid = fork();
    mem_addr("Global Variable", getpid(), (unsigned long)&a, &phy);
    mem_addr("Local Variable", getpid(), (unsigned long)&b, &phy);
    mem_addr("Local Static Variable", getpid(), (unsigned long)&c, &phy);
    mem_addr("Local Const", getpid(), (unsigned long)&d, &phy);
    sleep(1);
    free(p);
    waitpid();
    return 0;
}
```

③编译 address.c 文件，生成可执行文件 address，并执行：

```
[root@localhost ~]# gcc address.c -o address
```

[root@localhost ~]# ./address

分析上述程序的输出结果，并计算验证虚拟地址和物理地址是否正确，结合理论阐述过程。

10.3.3 动态分区存储管理模拟

动态分区分配策略包括：首次适应、循环首次适应、最佳适应和最坏适应，具体原理请查阅教材。在该实验中，将使用首次适应算法实现内存分配和回收的模拟，其他算法可自行实现。实验指导如下：

①为说明内存中哪些分区是空闲的，可以用来装入新的作业，需有一张空闲区表，如下图所示。其中，起始地址表示各空闲区的内存开始地址，长度表示空闲区的大小，状态未分配表示该条目是有效空闲区，状态空条目表示该条目没有登记信息或信息无效。由于分区个数不定，空闲区表中应有足够的空表目项。再设置一张已分配区表，记录作业或进程的内存占有情况。

起始地址	长 度	状 态
45K	20K	未分配
110K	146K	未分配
30K	10K	空条目
		空条目
80K	15K	空条目
		空条目

②当有一个新作业要求装入内存时，查空闲区表，从中找出一个大小能满足需要的空闲区，分割出一部分给作业，余下的仍留在空闲区表中。为方便查找，要不断对表进行整理，把“空条目”表项总是留在表的后部。找不到能满足需要的空闲区，则等待。

③当一个作业执行完成时，作业所占有的内存分区应收回。收回时应考虑相邻空闲区的合并问题。分 4 种情况处理：回收区下邻（低地址）空闲区；回收区上邻（高地址）空闲区；回收区上下都有邻接空闲区；回收区不与空闲区邻接。

④开始应对空闲区表和已分配区表进行初始化。学生自己设计一个作业申请队列和作业完成后的释放次序。

⑤每次作业申请和完成后，显示和打印相应的处理结果及空闲区表的变化情况。

⑥有兴趣的同学可增加空闲区“紧凑”功能。即当一个新作业要求装入内存

时，查空闲区表，找不到能满足需要的空闲区，但全部空闲区大小的总和能满足需求。这时，修改空闲区表和已分配区表，把分散的小空闲区归并成一个大空闲区，再进行分配。

实验操作过程：

①打开 Finalshell 程序，创建一个 C 程序文件，命名为 **partition.c** 并打开：

```
[root@localhost ~]# touch partition.c
[root@localhost ~]# vi partition.c
```

②按 **i** 键进入 **insert** 模式，在 partition.c 文件中输入如下代码，按 **Esc** 键退出 insert 模式，输入**:wq** 保存：

```
#include <stdio.h>
#define N 5
struct freearea {
    int startaddr;
    int size;
    int state;
} freeblock[N] = {{20,20,1},{80,50,1},{150,100,1},{300,30,0},{600,100,1}};

/*  定义分配主存空间函数  alloc() */
int alloc(int applyarea) {
    int i,tag=0;
    for(i=0; i<N; i++) {
        if(freeblock[i].state==1 && freeblock[i].size>applyarea) {
            freeblock[i].startaddr += applyarea;
            freeblock[i].size -= applyarea;
            tag = 1;
            return freeblock[i].startaddr-applyarea;
        } else if(freeblock[i].state==1 && freeblock[i].size==applyarea) {
            freeblock[i].state = 0;
            tag = 1;
            return freeblock[i].startaddr;
        }
    }
    if(tag == 0) return -1;
}

/*  定义主存空间回收函数  setfree() */
void setfree(){
    int s,l,tag1=0,tag2=0,tag3=0,i,j;
    printf("\nInput free area startaddress: ");
    scanf("%d",&s);//空闲区开始地址
    printf("\n Input free area size: ");
```

```
        scanf("%d",&l);//空闲区长度
        for(i=0; i<N; i++) {
            if(freeblock[i].startaddr==s+l && freeblock[i].state==1){
                l += freeblock[i].size;
                tag1 = 1;
                for(j=0; j<N; j++) {
                    if(freeblock[j].startaddr+freeblock[j].size== \
                       s && freeblock[j].state==1) {
                        freeblock[i].state = 0;
                        freeblock[j].size += l;
                        tag2 = 1;
                        break;
                    }
                }
                if(tag2 == 0){
                    freeblock[i].startaddr = s;
                    freeblock[i].size = l;
                    break;
                }
            }
        }
        if(tag1 == 0) {
            for(i=0; i<N; i++) {
                if(freeblock[i].startaddr+freeblock[i].size== \
                   s && freeblock[i].state==1) {
                    freeblock[i].size += l;
                    tag3 = 1;
                    break;
                }
            }
            if(tag3 == 0)
                for(j=0; j<N; j++)
                    if(freeblock[j].state == 0) {
                        freeblock[j].startaddr = s;
                        freeblock[j].size = l;
                        freeblock[j].state = 1;
                        break;
                    }
        }
    }
    void adjust() {
        int i,j;
        struct freearea middata;
        for(i=1; i<N; i++)//将空闲区按始地址顺序在表中排列
```

```
        for(j=0; j<N-i; j++)
            if(freeblock[j].startaddr > freeblock[j+1].startaddr) { //排序
                middata = freeblock[j];
                freeblock[j] = freeblock[j+1];
                freeblock[j+1] = middata;
            }
        for(i=1; i<N; i++)//将空表目放在表后面
            for(j=0; j<N-i; j++)
                if(freeblock[j].state == 0 && freeblock[j+1].state == 1) {
                    middata = freeblock[j];
                    freeblock[j] = freeblock[j+1];
                    freeblock[j+1] = middata;
                }
}
void print() {
    int i;
    printf(" |----------------------|\n");
    printf(" | startaddr size state |\n");
    for(i=0; i<N; i++)
        printf(" | %4d %4d %4d |\n",
            freeblock[i].startaddr,freeblock[i].size,freeblock[i].state);
}
int main() {
    int applyarea,start;
    long i;
    char end;
    printf("\nIs there any job request memory? (y or n): ");
    while( (end=getchar())=='y' ) {
        printf("At first the free memory is this:\n");
        adjust();
        print();
        printf("Input request memory size: ");
        scanf("%d",&applyarea);
        start = alloc(applyarea);
        if(start==-1) printf("There is no fit memory.please wait!!\n");
        else { printf("Job's memory start address is: %d\n",start);
            printf(" Job size is: %d\n",applyarea);
            printf("After allocation the free memory is this:\n");
            adjust();
            print();
            printf("Job is running.\n");
            for(i=0; i<100000; i++);
                printf("Job is terminated.\n");
        }
```

```
            setfree();
            adjust();
            print();
            printf("Is there any job that is waiting? (y or n): ");
            end = getchar();
        }
}
```

③编译 partition.c 文件，生成可执行文件 partition，并执行：

```
[root@localhost ~]# gcc partition.c -o partition
[root@localhost ~]# ./partiton
```

分析不同的分配和回收结果，并在此基础上实现其他的分配算法。

10.3.4 页面置换算法模拟

设计一个虚拟存储区和内存工作区，并使用下述算法计算访问命中率：

①最佳淘汰算法（OPT）

②先进先出的算法（FIFO）

③最近最久未使用算法（LRU）

④最不经常使用算法（LFU）

⑤最近未使用算法（NUR）

命中率＝1－页面失效次数 / 页地址流长度。实验指导如下：

首先，用 srand( )和 rand( )函数定义和产生指令序列，然后将指令序列变换成相应的页地址流，并针对不同的算法计算出相应的命中率。

①通过随机数产生一个指令序列，共 320 条指令。指令的地址按下述原则生成：

A：50%的指令是顺序执行的

B：25%的指令是均匀分布在前地址部分

C：25%的指令是均匀分布在后地址部分

具体的实施方法：

A：在[0，319]的指令地址之间随机选取一起点 m

B：顺序执行一条指令，即执行地址为 m+1 的指令

C：在前地址[0, m+1]中随机选取一条指令并执行，该指令的地址为 m’

D：顺序执行一条指令，其地址为 m’+1

E：在后地址[m’+2，319]中随机选取一条指令并执行

F：重复步骤 A-E，直到 320 次指令

②将指令序列变换为页地址流

设：页面大小为 1K；用户内存容量 4 页到 32 页；用户虚存容量为 32K。

在用户虚存中，按每 K 存放 10 条指令排列虚存地址，即 320 条指令在虚存中的存放方式为：

第 0 条-第 9 条指令为第 0 页（对应虚存地址为[0，9]）

第 10 条-第 19 条指令为第 1 页（对应虚存地址为[10，19]）

……

第 310 条-第 319 条指令为第 31 页（对应虚存地址为[310，319]）

按以上方式，用户指令可组成 32 页。

实验操作过程：

①打开 Finalshell 程序，创建一个 C 程序文件，命名为 **replacement.c** 并打开：

```
[root@localhost ~]# touch replacement.c
[root@localhost ~]# vi replacement.c
```

②按 **i** 键进入 **insert** 模式，在 replacement.c 文件中输入如下代码，按 **Esc** 键退出 insert 模式，输入**:wq** 保存：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define TRUE 1
#define FALSE 0
#define INVALID -1
#define total_instruction 320 /*指令流长*/
#define total_vp 32 /*虚页长数*/
#define clear_period 50 /*清 0 周期*/
typedef struct /*页面结构*/
{
  int pn,pfn,counter,time;//页号，页面框架号，计数器，时间
}pl_type;//页表项
pl_type pl[total_vp]; /*页面结构数组*/
struct pfc_struct{ /*页面控制结构*/
   int pn,pfn;
   struct pfc_struct *next;
};//页表的链式存储方式
typedef struct pfc_struct pfc_type;
pfc_type pfc[total_vp],*freepf_head,*busypf_head,*busypf_tail;
int diseffect, a[total_instruction];//页面失效次数，指令流数据组
```

```
int page[total_instruction], offset[total_instruction];
//每条指令所属的页号；每页装入 10 条指令后取模运算页号偏移
int initialize(int);
int FIFO(int);
int LRU(int);
int LFU(int);
int NUR(int);
int OPT(int);
int main( )
{
  int s,i,j;
  srand(10*getpid());
  s=(float)319*rand( )/RAND_MAX +1; //
  for(i=0;i<total_instruction;i+=4) /*产生指令队列*/
  {
       if(s<0||s>319)
       {
          printf("When i==%d,Error,s==%d\n",i,s);
          exit(0);
         }
a[i]=s; /*任选一指令访问点 m*/
       a[i+1]=a[i]+1; /*顺序执行一条指令*/
       a[i+2]=(float)a[i]*rand( )/RAND_MAX; /*执行前地址指令 m' */
       a[i+3]=a[i+2]+1; /*顺序执行一条指令*/
       s=(float)(318-a[i+2])*rand( )/RAND_MAX +a[i+2]+2;
       if((a[i+2]>318)||(s>319))
          printf("a[%d+2],a number which is :%d and s==%d\n",i,a[i+2],s);
    }
for (i=0;i<total_instruction;i++) /*将指令序列变换成页地址流*/
{
          page[i]=a[i]/10;
offset[i]=a[i]%10;
      }
     for(i=4;i<=32;i++) /*用户内存工作区从 4 个页面到 32 个页面*/
    {
          printf("---%2d page frames---\n",i);
          FIFO(i);
          LRU(i);
          LFU(i);
          NUR(i);
          OPT(i);
     }
        return 0;
  }
```

```
int initialize(total_pf) /*初始化相关数据结构*/
int total_pf;//分配给进程的物理块数  /*用户进程的内存页面数*/
{int i;
 diseffect=0;
 for(i=0;i<total_vp;i++)//有 32 页，对每页进行循环，初始化页表
 {
    pl[i].pn=i;//页号
    pl[i].pfn=INVALID; //物理块号  /*置页面控制结构中的页号，页面为空*/
    pl[i].counter=0;
    pl[i].time=-1; /*页面控制结构中的访问次数为 0，时间为-1*/
 }
 for(i=0;i<total_pf-1;i++)
 {
     pfc[i].next=&pfc[i+1];
     pfc[i].pfn=i;
  } /*建立 pfc[i-1]和 pfc[i]之间的链接*///把可用的几个物理块前后链接起来
 pfc[total_pf-1].next=NULL;//最后一个物理块要单独处理
 pfc[total_pf-1].pfn=total_pf-1;
 freepf_head=&pfc[0]; /*空页面队列的头指针为 pfc[0]*/
 //开始的时候所有的物理块都是空的
 return 0;
 }
int FIFO(total_pf) /*先进先出算法*/
int total_pf; /*用户进程的内存页面数*/
{
    int i,j;
    pfc_type *p;
    initialize(total_pf); /*初始化相关页面控制用数据结构*/
    busypf_head=busypf_tail=NULL;
    for(i=0;i<total_instruction;i++)//对所有指令进行循环，模式顺序执行所有指令
{
     if(pl[page[i]].pfn==INVALID)
     {//如果该指令对应的页还没有分配物理块的话，下面将调入内存
         diseffect+=1; /*失效次数*/
         if(freepf_head==NULL) /*无空闲页面*///空闲页面指的是空闲物理块？
         {
            p=busypf_head->next;//忙内存块
            pl[busypf_head->pn].pfn=INVALID;//换出去，则以为着没有分配
            freepf_head=busypf_head; /*释放忙页面队列的第一个页面*/
            freepf_head->next=NULL;//只释放一个页面，故只有一个元素
            busypf_head=p;//忙队列头指针向后移动一个位置
         }
         p=freepf_head->next; /*按 FIFO 方式调新页面入内存页面*/
         freepf_head->next=NULL;
```

```
            freepf_head->pn=page[i];
            pl[page[i]].pfn=freepf_head->pfn;
            if(busypf_tail==NULL)//被占用一个了，所以忙队列不为空了
              busypf_head=busypf_tail=freepf_head;
            else
            {
               busypf_tail->next=freepf_head; /*free 页面减少一个*/
               busypf_tail=freepf_head;
            }
            freepf_head=p;
         }
      }
    printf("FIFO:%6.4f\n",1-(float)diseffect/320);
    return 0;
}
int LRU (total_pf) /*最近最久未使用算法*/
int total_pf;
{
  int min,minj,i,j,present_time;
  initialize(total_pf);
  present_time=0;
  for(i=0;i<total_instruction;i++)
  {
    if(pl[page[i]].pfn==INVALID) /*页面失效*/
    {
       diseffect++;
       if(freepf_head==NULL) /*无空闲页面*/
{
          min=32767;
          for(j=0;j<total_vp;j++) /*找出 time 的最小值*/
             if(min>pl[j].time&&pl[j].pfn!=INVALID)
             {
                    min=pl[j].time;
                    minj=j;
          }
          freepf_head=&pfc[pl[minj].pfn]; //腾出一个单元
          pl[minj].pfn=INVALID;
          pl[minj].time=-1;
          freepf_head->next=NULL;
       }
       pl[page[i]].pfn=freepf_head->pfn; //有空闲页面,改为有效
       pl[page[i]].time=present_time;
       freepf_head=freepf_head->next; //减少一个 free  页面
     }
```

```
     else
        pl[page[i]].time=present_time; //命中则增加该单元的访问次数
     present_time++;
   }
  printf("LRU:%6.4f\n",1-(float)diseffect/320);
  return 0;
}
int NUR(total_pf) /*最近未使用算法*/
int total_pf;
{ int i,j,dp,cont_flag,old_dp;
   pfc_type *t;
   initialize(total_pf);
   dp=0;
   for(i=0;i<total_instruction;i++)
   { if (pl[page[i]].pfn==INVALID) /*页面失效*/
      {diseffect++;
       if(freepf_head==NULL) /*无空闲页面*/
       { cont_flag=TRUE;
          old_dp=dp;
          while(cont_flag)
          if(pl[dp].counter==0&&pl[dp].pfn!=INVALID)
          cont_flag=FALSE;
          else
          {
               dp++;
               if(dp==total_vp)
                  dp=0;
               if(dp==old_dp)
                           for(j=0;j<total_vp;j++)
                             pl[j].counter=0;
          }
          freepf_head=&pfc[pl[dp].pfn];
          pl[dp].pfn=INVALID;
          freepf_head->next=NULL;
        }
       pl[page[i]].pfn=freepf_head->pfn;
       freepf_head=freepf_head->next;
       }
      else
      pl[page[i]].counter=1;
      if(i%clear_period==0)
      for(j=0;j<total_vp;j++)
         pl[j].counter=0;
   }
```

```
printf("NUR:%6.4f\n",1-(float)diseffect/320);
return 0;
}
int OPT(total_pf) /*最佳置换算法*/
int total_pf;
{ int i,j, max,maxpage,d,dist[total_vp];
pfc_type *t;
initialize(total_pf);
for(i=0;i<total_instruction;i++)
{ //printf("In OPT for 1,i=%d\n",i);
//i=86;i=176;206;250;220,221;192,193,194;258;274,275,276,277,278;
      if(pl[page[i]].pfn==INVALID) /*页面失效*/
      {
         diseffect++;
         if(freepf_head==NULL) /*无空闲页面*/
         {for(j=0;j<total_vp;j++)
          if(pl[j].pfn!=INVALID) dist[j]=32767; /*  最大"距离" */
          else dist[j]=0;
          d=1;
          for(j=i+1;j<total_instruction;j++)
          {
             if(pl[page[j]].pfn!=INVALID)
             dist[page[j]]=d;
             d++;
          }
max=-1;
            for(j=0;j<total_vp;j++)
            if(max<dist[j])
          {
             max=dist[j];
             maxpage=j;
          }
          freepf_head=&pfc[pl[maxpage].pfn];
          freepf_head->next=NULL;
          pl[maxpage].pfn=INVALID;
         }
  pl[page[i]].pfn=freepf_head->pfn;
       freepf_head=freepf_head->next;
   }
}
printf("OPT:%6.4f\n",1-(float)diseffect/320);
return 0;
}
int LFU(total_pf) /*最不经常使用置换法*/
```

```
int total_pf;
{
   int i,j,min,minpage;
   pfc_type *t;
initialize(total_pf);
   for(i=0;i<total_instruction;i++)
   { if(pl[page[i]].pfn==INVALID) /*页面失效*/
     { diseffect++;
       if(freepf_head==NULL) /*无空闲页面*/
       { min=32767;
         for(j=0;j<total_vp;j++)
         {if(min>pl[j].counter&&pl[j].pfn!=INVALID)
           {
         min=pl[j].counter;
                 minpage=j;
            }
pl[j].counter=0;
       }
      freepf_head=&pfc[pl[minpage].pfn];
      pl[minpage].pfn=INVALID;
      freepf_head->next=NULL;
   }
   pl[page[i]].pfn=freepf_head->pfn; //有空闲页面,改为有效
   pl[page[i]].counter++;
   freepf_head=freepf_head->next; //减少一个 free  页面
  }
}
  else
  pl[page[i]].counter++;
  }
printf("LFU:%6.4f\n",1-(float)diseffect/320);
return 0;
}
```

③编译 replacement.c 文件，生成可执行文件 replacement 并执行：

```
root@localhost ~]# gcc replacement.c -o replacement
[root@localhost ~]# ./replacement
```

10.4 实验讨论

虚拟地址长度在实际使用时为什么是 48 位，修改本实验的程序测试 Windows 系统的地址长度信息。此外，页面置换有没有更好的算法，请自行设计和讨论。